czii-cryo-et-object-identification

9th place solution

Rank	9
Team	Avengers
Author	NANACHI
Topic ID	561431
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561431
Retrieved with Kaggle CLI on 2026-10-21.

First of all, I would like to express my sincere gratitude to the competition host and the Kaggle staff for organizing such a fascinating competition. I thoroughly enjoyed this competition and learned a great deal in the process!

Furthermore, I'd like to thank @hengck23 and @davidlist . @hengck23 's discussion and notebook were the starting point for my solution, and Many of the discussions and comments from @davidlist were extremely helpful.

Summary

My solution is simple and straightforward. I performed segmentation using a 3D ConvNeXt-like model and further employed an ensemble of as many models as possible. Subsequently, I computed the centroids of the particles using cc3d and curated the predicted centroids with DBSCAN.

Segmentation Mask

I used ground truth masks with an adjusted radius for each particle. As a result, the public leaderboard score jumped 0.02~0.04. Concletely, I adjust mask size as shown in the below table;

apo-ferritin(r=60)	beta-galactosidase(r=90)	ribosome(r=150)	thyroglobulin(r=130)	virus-like-particle(r=135)
60/2	90/2	150/3	130/3	135/3

Since the competition metric requires that the predicted centroid falls within a radius of r×0.5 from the ground truth centroid, I believe that multiplying the radius of each particle by a factor of 0.5 or less is reasonable.

Model Architecture

In this section, I explain my model architecture both encoder and decoder.

Encoder

To implement the encoder, I started with the ConvNeXt, since ConvNeXt is very powerful and fast. Then, I customized the model to suit the task that is to predict small particles. These changes from the original ConvNeXt are below;

- 2D to 3D for all convolutional layer
- 4x4 stem -> 2x2 stem. Because the particles are very small, I believed that a large stem would adversely affect the model's predictions, especially apo-ferritin and beta-galactosidase. This modification slowed down the model's inference, but it improved its cross-validation performance.
- 7x7 kernel size in conv block ->3x3. The reason why I modified like this is the same as the stem. This modification improved both inference speed and cv.
- (3, 3, 9, 3) number of block -> (3, 3, 3, 3). this modification is to reduce a number of parameters and get more inference speed. cv did not decrease.

Decoder

The flow of the decoder is based on U-Net. The implementation of conv block is shown in the following code;

```
 (nn.Module):
     ():
        ().__init__()
        .conv1 = nn.Conv3d(in_channels, in_channels, kernel_size=, padding=, bias=,
groups=in_channels)
        .norm1 = nn.GroupNorm(num_groups=, num_channels=in_channels)
        .act1 = nn.GELU()
        .conv2 = nn.Conv3d(in_channels, in_channels*, kernel_size=, bias=)
        .conv3 = nn.Conv3d(in_channels*, out_channels, kernel_size=, bias=)

     ():

        x = .conv1(x)
        x = .norm1(x)
        x = .conv2(x)
        x = .act1(x)
        x = .conv3(x)
         x
```

And the implementation of the decoder is shown in the following code;

```
 (nn.Module):
     ():
        ().__init__()
        .up3 = nn.Upsample(scale_factor=(,,), mode=, align_corners=)

        .dec3 = ConvBlock3D(in_channels=encoder_dims[] + encoder_dims[], out_channels=decoder_dims[])
       ...

     ():
        x, f0, f1, f2, f3 = features


        d3 = .up3(f3)
        d3 = torch.cat([d3, f2], dim=)
        d3 = .dec3(d3)
        ...
```

Since the ground truth for segmentation is simply a sphere, I used a basic `nn.Upsample` for upsampling instead of `nn.ConvTranspose3d` to reduce parameters.

Other Details

- loss dunction is bce
- My inference code is based on @hengck 's this notebook and the idea of DBScan came from @linheshen 's notebook.
- The window size for inference is (32, 320, 320) (overwrap of z axis is 6). The window size for training is (32, 256, 256).
- TTAs are rot90, 180 and 270.
- preprocessing is only normalization

```
():
    lower, upper = np.percentile(x, (, ))
    x = np.clip(x, lower, upper)
    x = x - np.(x)
    x = x / np.(x)
     x
```

- Augmentations for training are rot90, 180, 270, and xyz flip.

What did not work

- pretraining with synthetic data provided by host
- more and more smaller mask(e.g. factor=0.25 for large particles and factor=0.4 for small particles)
- flip tta instead of rot90

Code & Model Checkpoint

inference code: https://www.kaggle.com/code/wadakoki/czii-9th-inference
training code: https://colab.research.google.com/drive/1OG5Rap9zUkKG6fnGZ8kP0JwwFZfSTgCc?usp=sharing
model checkpoint: https://www.kaggle.com/datasets/wadakoki/czii-9th-final-models/data
generated mask: https://www.kaggle.com/datasets/wadakoki/czii-mask-small
code for mask generation: https://www.kaggle.com/code/wadakoki/czii-gen-small-masks

Requirements for training code on google colab

- GPU: L4
- RAM: High Memory